第 4 章 · 教学源码

四组示例的完整代码。正式源文件位于本章 `src/main/java/`，本页由这些文件生成，便于浏览器直接阅读；编译与运行方法见 运行说明。

CH04-E01 · CH04-E02 · CH04-E03 · CH04-E04

CH04-E01 · 泛型、成员与异常处理

源文件：`src/main/java/org/example/jvminternals/ch04/structure/ClassfileSubject.java`

```java
package org.example.jvminternals.ch04.structure;

/**
 * CH04-E01：供反汇编阅读的泛型类；不承担校验或故障复现职责。
 * 泛型上界令字段描述符与泛型签名呈现不同的信息。
 *
 * @param <T> 数值类型
 */
public final class ClassfileSubject<T extends Number> {
    public static final String LABEL = "classfile";
    private final T value;

    /** 保存数值，供泛型字段与访问方法的结构观察使用。 */
    public ClassfileSubject(T value) {
        this.value = value;
    }

    /** 返回数值；描述符中的擦除类型为 Number。 */
    public T value() {
        return value;
    }

    /**
     * 把输入转为整数，空输入或非整数输入采用字段值。
     * 分支和异常处理用于展示 Code、异常表及栈映射之间的关系。
     */
    public int parseOrDefault(String input) {
        if (input == null) {
            return value.intValue();
        }
        try {
            return Integer.parseInt(input);
        } catch (NumberFormatException exception) {
            // 输入不符合整数格式属于本方法的正常回退契约。
            return value.intValue();
```

```
        }
    }
}
```

CH04-E02 · 现代声明与反射

源文件：`src/main/java/org/example/jvminternals/ch04/modern/ModernShapes.java`

```
package org.example.jvminternals.ch04.modern;

import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;
import java.util.Arrays;

/** CH04-E02：将现代类声明、嵌套关系与反射可见信息放在同一组产物中观察。 */
public final class ModernShapes {
    private static final System.Logger LOGGER =
System.getLogger(ModernShapes.class.getName());
    private final int offset;

    /** 创建外部实例；非静态成员类会保留对该实例的引用。 */
    public ModernShapes(int offset) {
        this.offset = offset;
    }

    /** 运行期保留的类型注解，供反射读取。 */
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.TYPE)
    public @interface RuntimeTag {
    }

    /** 只保留在类文件中的类型注解，普通运行期注解查询不可见。 */
    @Retention(RetentionPolicy.CLASS)
    @Target(ElementType.TYPE)
    public @interface ClassTag {
    }

    /** 密封接口的允许实现集合写入 PermittedSubclasses 属性。 */
    public sealed interface Shape permits Point {
    }

    /** 记录组件由 Record 属性描述，访问器等仍以普通方法表示。 */
    @RuntimeTag
    @ClassTag
```

```
    public record Point(int x, int y) implements Shape {
    }

    /** 非静态成员类访问外部私有字段，用于解释内部类和巢成员的不同职责。 */
    public final class Member {
        /** 返回外部实例字段；它与巢主关系及外部实例引用分别有关。 */
        public int offset() {
            return ModernShapes.this.offset;
        }
    }

    /** 输出反射元数据；不比较预期值、不包含断言。 */
    public static void main(String[] args) {
        LOGGER.log(System.Logger.Level.INFO, "开始读取记录类、密封接口与注解元数据");
        System.out.println("Point.isRecord=" + Point.class.isRecord());
        System.out.println("Point.recordComponents=" +
Arrays.toString(Point.class.getRecordComponents()));
        System.out.println("Point.annotations=" +
Arrays.toString(Point.class.getDeclaredAnnotations()));
        System.out.println("Shape.isSealed=" + Shape.class.isSealed());
        System.out.println("Shape.permitted=" +
Arrays.toString(Shape.class.getPermittedSubclasses()));
        System.out.println("Member.nestHost=" + Member.class.getNestHost().getName());
        LOGGER.log(System.Logger.Level.INFO, "元数据读取结束");
    }
}
```

CH04-E03 · 类文件模型读取

源文件：`src/main/java/org/example/jvminternals/ch04/model/ClassfileExplorer.java`

```
package org.example.jvminternals.ch04.model;

import java.io.IOException;
import java.lang.classfile.ClassFile;
import java.lang.classfile.ClassModel;
import java.nio.file.Path;

/** CH04-E03：读取指定类文件，按字段、方法和属性层级输出不可变模型。 */
public final class ClassfileExplorer {
    private static final System.Logger LOGGER =
System.getLogger(ClassfileExplorer.class.getName());

    private ClassfileExplorer() {
    }
```

```
    /**
     * 接收一个本地类文件路径；输入错误由进程入口传播并显示原因。
     * 只访问模型，不定义类、不运行目标方法，也不调用校验器。
     */
    public static void main(String[] args) throws IOException {
        if (args.length != 1) {
            throw new IllegalArgumentException("用法: ClassfileExplorer <类文件路径>");
        }
        LOGGER.log(System.Logger.Level.INFO, "开始读取类文件模型");
        ClassModel model = ClassFile.of().parse(Path.of(args[0]));
        System.out.printf("class=%s version=%d.%d%n", model.thisClass().asInternalName(),
                model.majorVersion(), model.minorVersion());
        for (var field : model.fields()) {
            System.out.printf("field=%s descriptor=%s%n", field.fieldName().stringValue(),
                    field.fieldType().stringValue());
        }
        for (var method : model.methods()) {
            System.out.printf("method=%s descriptor=%s%n",
method.methodName().stringValue(),
                    method.methodType().stringValue());
            method.code().ifPresent(code -> {
                for (var element : code) {
                    System.out.printf("  code-element=%s%n", element);
                }
            });
        }
        for (var attribute : model.attributes()) {
            System.out.println("class-attribute=" +
attribute.attributeName().stringValue());
        }
        LOGGER.log(System.Logger.Level.INFO, "类文件模型读取结束");
    }
}
```

CH04-E04 · 生成与变换

源文件：`src/main/java/org/example/jvminternals/ch04/transform/ClassfileWorkshop.java`

```
package org.example.jvminternals.ch04.transform;

import java.io.IOException;
import java.lang.classfile.ClassFile;
import java.lang.classfile.ClassTransform;
import java.lang.classfile.Label;
import java.lang.classfile.instruction.ConstantInstruction;
import java.lang.constant.ClassDesc;
```

```
import java.lang.constant.ConstantDescs;
import java.lang.constant.MethodTypeDesc;
import java.nio.file.Files;
import java.nio.file.Path;

/** CH04-E04：生成带分支的方法，再用元素流变换替换两个整数常量。 */
public final class ClassfileWorkshop {
    private static final System.Logger LOGGER =
System.getLogger(ClassfileWorkshop.class.getName());
    private static final String CLASS_NAME = "GeneratedNumber.class";

    private ClassfileWorkshop() {
    }

    /**
     * 将原始和变换后的产物写入不同子目录；已有文件不覆盖。
     * 程序只生成产物供工具阅读，不加载产物或判断其业务正确性。
     */
    public static void main(String[] args) throws IOException {
        if (args.length != 1) {
            throw new IllegalArgumentException("用法：ClassfileWorkshop <新的输出目录>");
        }
        Path output = Path.of(args[0]);
        Files.createDirectory(output);
        Path originalDirectory = Files.createDirectory(output.resolve("original"));
        Path transformedDirectory = Files.createDirectory(output.resolve("transformed"));
        // 有分支时显式生成栈映射；不要靠手改指令偏移复制旧帧。
        ClassFile classFile = ClassFile.of(ClassFile.StackMapsOption.GENERATE_STACK_MAPS);
        LOGGER.log(System.Logger.Level.INFO, "开始生成具有两个返回分支的类文件");
        byte[] original = classFile.build(
                ClassDesc.of("org.example.jvminternals.ch04.generated.GeneratedNumber"),
                builder -> builder.withFlags(ClassFile.ACC_PUBLIC | ClassFile.ACC_FINAL)
                        .withMethodBody("value", MethodTypeDesc.of(ConstantDescs.CD_int,
ConstantDescs.CD_boolean),
                                ClassFile.ACC_PUBLIC | ClassFile.ACC_STATIC, code -> {
                                    Label negative = code.newLabel();
                                    code.iload(0).ifeq(negative).bipush(7).ireturn()
                                            .labelBinding(negative).bipush(-7).ireturn();
                                }));
        Files.write(originalDirectory.resolve(CLASS_NAME), original);
        byte[] transformed = classFile.transformClass(classFile.parse(original),
                ClassTransform.transformingMethodBodies((builder, element) -> {
                    if (element instanceof ConstantInstruction constant
                            && constant.constantValue() instanceof Integer value) {
                        // 本例生成的方法只包含这两个整数；其他元素原样转交。
                        builder.bipush(value > 0 ? 9 : -9);
                    } else {
                        builder.with(element);
                    }
                }));
        Files.write(transformedDirectory.resolve(CLASS_NAME), transformed);
```

```
        System.out.printf("original=%s bytes=%d%n", originalDirectory.resolve(CLASS_NAME),
original.length);
        System.out.printf("transformed=%s bytes=%d%n",
transformedDirectory.resolve(CLASS_NAME), transformed.length);
        LOGGER.log(System.Logger.Level.INFO, "生成与变换结束，产物未加载执行");
    }
}
```